



Manual de uso

Versão 3.4.0 · setembro de 2026
WINDEV · WEBDEV · WINDEV Mobile

Manual de uso — WX Claude Code

Plugin do Claude Code para converter um projeto **WINDEV**, **WEBDEV** ou **WINDEV Mobile** para outra linguagem sem inventar o que o projeto faz. Este manual é a sequência de uso, do zero até o corte, com o comando de cada passo.

1. Instalar

Requisitos: Claude Code, Python 3.10 ou mais novo, Node 22 ou mais novo (só para o Impeccable).

```
claude plugin marketplace add adrianoboller/adrianoboller
claude plugin install wx-claude-code@wx-claude-code
```

Para testar sem instalar, a partir da pasta do repositório:

```
claude --plugin-dir ./wx-claude-code
```

Confira se carregou: numa sessão nova, peça «liste as skills e os agentes com prefixo `wx-claude-code:` ». Devem aparecer cinco comandos, três skills e 34 agentes.

2. Preparar os anexos

Crie uma pasta de evidências (por exemplo `inputs/`) dentro do projeto de destino e coloque nela, com nomes claros:

ITEM	O QUE É	COMO GERAR NO WX
<code>.SQL</code>	DDL do banco: tabelas, índices, constraints, triggers	análise → exportar script SQL, ou dump do HFSQL
PDF dos códigos	só procedures, classes e eventos	documentação técnica → filtrar código
PDF das interfaces	só janelas, páginas, controles e relatórios	documentação técnica → filtrar telas
PDF das queries	só as queries: nome, SQL, parâmetros	documentação técnica → filtrar queries
PDF completo	a documentação técnica inteira	documentação técnica → tudo
Screenshots	cada tela em cada estado (normal, vazio, erro)	capturas identificadas

Os anexos são somente leitura: o plugin nunca escreve neles. Sem PDF pesquisável (texto) a extração exige OCR, e o pré-flight marca isso.

3. Responder o questionário (A–J)

```
/wx-claude-code:questionario ./meu-projeto
```

O plugin pergunta **uma letra por mensagem** e espera. A resposta decide a próxima: quem não tem o PDF de códigos (B) ouve, em E, que o completo vai cobrir; quem diz «não» ao Impeccable (F) não é perguntado sobre paleta; quem escolhe mobile (I) é perguntado sobre versões de Android e iOS. Cada resposta é confirmada em uma linha antes da letra seguinte.

As letras:

LETRA	PERGUNTA	RESPOSTA TÍPICA
A	caminho do <code>.SQL</code> , dialeto, versão, encoding, collation	<code>inputs/banco.sql</code> , HFSQL 2025, utf-8
B	PDF só dos códigos, pesquisável?	<code>inputs/codigo.pdf</code> , sim
C	PDF só das interfaces	<code>inputs/telas.pdf</code>
D	PDF só das queries	<code>inputs/queries.pdf</code>
E	PDF completo	<code>inputs/completo.pdf</code>
F	estilo das telas com o Impeccable: paleta, tema, tipografia, densidade, preservar ou redesenhar	sim, <code>#E2261C</code> sobre <code>#010418</code> , Exo 2, compacta, redesenhar
G	usar o corpus do Help WLanguage 12k? override da versão?	sim, versão 2025
H	para qual linguagem converter o backend; se não souber, o plugin pergunta quatro sinais e mostra Rust, Python e C# + WL_C# com o porquê	C# + WL_C# (equipe WINDEV, desktop, prazo) ou Rust + Axum (desempenho)
I	frontend: linguagem, framework, plataformas	TypeScript + React, web
J	ativar economia de tokens?	sim

A orientação de linguagem está em `skills/conversao-wx/references/perfis-de-destino.md`, com o que cada perfil ganha, custa e para que projeto serve. O perfil **C# + WL_C#** usa a biblioteca de Bernard Sobra que porta mais de 480 funções do WLanguage com o mesmo nome; o `WL.dll` é baixado da release oficial e conferido por hash (`references/perfil-csharp-wl.md`).

E as três de governança: versão e idioma do WX, modo (`inventário`, `plano`, `piloto`, `completo`) e quem aprova.

Um caminho só conta como fornecido depois que o plugin **abre o arquivo**. Quem não tem um item responde «não tenho»: isso também é resposta, e vira `missing` no manifesto, nunca `not_applicable` por inferência.

O que sai:

<code>.wx-migration/</code>	
<code>questionario.json</code>	suas respostas
<code>wx-inputs.manifest.json</code>	manifesto que o pré-flight lê
<code>conversion.config.json</code>	modo, destino, fidelidade
<code>gaps.md, traceability.csv</code>	vazios, prontos para o G1
<code>CLAUDE.md</code>	regras do projeto (com estilo de resposta se J = sim)
<code>DESIGN.md</code>	esboço da paleta (se F = sim)

4. Converter por gates

```
/wx-claude-code:converter piloto ./meu-projeto
```

O comando roda o **Gate G0** (pré-flight): verifica cada anexo fisicamente, hash, assinatura de PDF, metadados, e devolve `READY`, `CONDITIONAL` ou `BLOCKED` com a lista do que falta. `BLOCKED` não escreve código: entrega inventário, perguntas e lacunas.

Depois, gate a gate, até o modo pedido:

GATE	O QUE ACONTECE	QUEM APROVA
G1	inventário, hashes, extração de texto, índice do Help	líder técnico
G2	regras, telas, dados, queries, integrações, conflitos	responsável de negócio
G3	arquitetura-alvo, ADRs, plano de ondas, rollback	arquitetura
G4	piloto vertical: uma fatia com tela, regra, query e erro	técnico + negócio
G5	ondas de implementação por módulo	líder técnico
G6	segurança, desempenho, concorrência, testes	qualidade
G7	ensaio, reconciliação, cutover e plano de retorno	patrocinador

Regras que valem em todos: conflito entre evidências para o item e pergunta; o Help é semântica técnica, nunca regra de negócio; quem implementa não aprova; o piloto (G4) nunca é pulado numa conversão completa.

Como o plugin consulta o Help

Cada símbolo WLanguage vai para o especialista do tema certo do corpus da PC SOFT (HFSQL, controles, comunicação, funções padrão, mobile, web, erros). O especialista lê só a fatia dele e devolve assinatura, parâmetros, retorno, efeitos e a página de origem com hash. Para consultar à mão:

```
python3 "$CLAUDE_PLUGIN_ROOT/skills/conversao-wx/scripts/query_wlanguage_help.py" \
--query HReadSeekFirst --group 01-03-03 --version 2025 --limit 5
```

Como o plugin escolhe o modelo

Antes de cada delegação o orquestrador chama `rotear_modelo.py` com a classe da tarefa (`mecanica`, `analise`, `decisao`, `revisao`) e os sinais de risco. Haiku faz o mecânico, Sonnet analisa e implementa, Opus decide e revisa. Conflito, fiscal ou dado pessoal sobem um degrau; padrão já aprovado desce; acima de 80 % do orçamento do gate rebaixa, acima de 100 % bloqueia e o PMO decide. Tudo fica em `.wx-migration/pmo/roteamento.jsonl`.

Extrair o texto dos PDFs e capturar o golden master

```
python3 "$P/extrair_pdf.py" --manifest .wx-migration/wx-inputs.manifest.json --allowed-evidence-root ./inputs --output .wx-migration/evidence/pdf-text
python3 "$P/golden.py" capturar --casos inputs/dados-de-amostra/resultados-esperados.json --saida .wx-migration/tests/golden-master/casos.json
python3 "$P/golden.py" comparar --golden .wx-migration/tests/golden-master/casos.json --comando "python3 novo/regras.py" --relatorio .wx-migration/tests/results/piloto.json
```

(`$P` é `$CLAUDE_PLUGIN_ROOT/skills/conversao-wx/scripts`.) A extração grava uma página por arquivo com `arquivo#page=N` e o hash do PDF. O golden compara cada caso com tolerância e devolve `equivalência: n/total`; é esse número que o piloto apresenta.

O portão do G0

Enquanto o último pré-flight estiver `BLOCKED`, o hook do plugin nega qualquer escrita fora de `.wx-migration/`. Não é regra para o modelo obedecer: é o harness recusando. Resolva os anexos ou registre as lacunas e rode o G0 de novo.

5. Gerir o projeto (PMO)

```
/wx-claude-code:pmo iniciar ./meu-projeto
```

Cria `.wx-migration/pmo/` com plano por gate, orçamento, RAID, backlog, quadro e base de conhecimento. Daí em diante:

Scrum. Uma sprint por gate ou onda:

```
pmo.py sprint abrir --nome "Piloto de vendas" --objetivo "..." --gate G4 --item B
R-001 --item QRY-001
pmo.py sprint fechar --decisao APPROVED|CONDITIONAL|REJECTED --pedido "..."
```

O fechamento escreve o resumo de doze seções em `pmo/sprints/` e devolve ao backlog o que não atingiu a definição de pronto (evidência, implementação, teste, resultado comparado, aprovação humana, confiança nunca `low`).

Kanban. `pmo.py kanban` gera o quadro da matriz de rastreabilidade com limite de WIP (padrão 6 em andamento, 4 em verificação). Coluna estourada aparece marcada e não recebe cartão novo. O quadro não se edita: muda-se o estado na matriz.

PDCA. Toda hipótese de trabalho abre um ciclo com critério numérico:

```
pmo.py pdca abrir --gate G4 --hipotese "..." --medida "..." --criterio "ganho >=
1,5x"
pmo.py pdca fechar --id PDCA-001 --resultado infrutifero --medido "1,06x" --apren
dizado "..." --proxima "..."
```

O fechamento grava uma linha em `pmo/base_de_conhecimento.md` **nos dois casos**. A recusa com o número é resultado tão válido quanto o ganho, e é o que impede a mesma ideia de voltar sem medição. Infrutífero sem a próxima hipótese não fecha.

Uso de tokens medido. `uso_de_tokens.py --project-root . lancar --gate G4` lê o campo `usage` das sessões do Claude Code e lança no orçamento; `pmo.py gastar` fica para lançamento manual.

Painel. `pmo.py painel` gera `pmo/painel.html` para abrir no navegador. `pmo.py status` regenera `pmo/status.md`: gates, itens por estado, lacunas, decisões pendentes, orçamento por modelo, sprint, quadro e PDCA. Nenhum número é digitado; cada linha diz a fonte; o que não tem fonte é `INDISPONÍVEL`, nunca zero.

6. Estilo das telas

```
/wx-claude-code:estilo-telas ./meu-projeto
```

Usa a resposta F e o Impeccable para escrever `PRODUCT.md` e `DESIGN.md`: tokens de cor com contraste medido (mínimo 4,5:1 em texto), tema, tipografia com fallback real, densidade, e a convenção das cores de ação (verde inclui, amarelo altera, vermelho exclui, azul consulta; sempre contorno). Cada tela convertida passa por `/impeccable polish` antes de ser dada como pronta e cada onda por `/impeccable audit`.

7. Economia de tokens

```
/wx-claude-code:laudo-tokens
```

Três fases. A primeira é somente leitura e termina numa tabela de problemas por impacto; então **para** e espera o seu OK. A segunda propõe uma mudança por vez. A terceira entrega até três hábitos, só os que tiverem evidência nas suas sessões. Todo número é `MEDIDO`, `ESTIMADO` ou `INDISPONÍVEL`.

8. Onde fica cada coisa

```
.wx-migration/  
  questionario.json, wx-inputs.manifest.json, conversion.config.json  
  preflight/runs/<run>/      report.md, inventory.csv, gaps.md  
  evidence/                  hashes, texto extraído, índices  
  specifications/            regras, telas, dados, integrações  
  architecture/adr/          decisões de arquitetura  
  decisions/DEC-*.md         decisões humanas  
  gaps.md                    lacunas GAP-*  
  traceability.csv           evidência → regra → decisão → código → teste → result  
ado  
  tests/golden-master/       resultados do legado para comparar  
  pmo/                       plano, orçamento, RAID, backlog, kanban, pdca/, base_  
de_conhecimento.md, sprints/, status.md  
  logs/                      saídas longas, citadas por localizador
```

8a. Projeto de exemplo

`exemplos/estoque-wx/` tem tudo que o questionário pede, já respondido. Rode nele os passos 3 e 4 para ver o fluxo inteiro antes de usar um projeto seu.

9. Problemas comuns

- **«BLOCKED» no G0.** Leia `preflight/runs/<run>/report.md`: cada erro diz o grupo e o arquivo. PDF sem `page_count` ou sem `%%EOF` é PDF que o plugin não conseguiu abrir de verdade.
- **Skill não aparece na sessão.** Descrição de skill acima de 300 caracteres some da listagem quando o plugin inteiro carrega; o validador avisa.
- **Corpus com hash divergente.** Não use o corpus; o plugin marca `wlanguage_help_json` como bloqueado. O zip certo tem 26.750.976 bytes.
- **Orçamento estourado.** `rotear_modelo.py` devolve `BLOQUEADO`; decida no PMO com o número, não com adjetivo: aumentar a previsão, reduzir o escopo da sprint ou rebaixar tarefas.
- **Ciclo PDCA infrutífero não fecha.** Faltou `--proxima`. Hipótese que morre gera a próxima.

10. O que o plugin não faz

Não lê o formato binário do WX, não faz OCR sozinho, não certifica LGPD, não aprova gate no lugar do humano e não afirma equivalência sem baseline executável do legado.